rag -- 2026-10-30

5 new paper(s) · topic: Retrieval-augmented generation: retrieval quality, grounding and faithfulness, evaluation benchmarks, and RAG system design.

Worth reading in full

[P3] Decompose, Estimate, Select: Coverage-Aware Context Construction for Long-Form RAG -- Paper [P3] introduces a comprehensive coverage-aware retrieval pipeline and a grounded self-feedback loop that directly address grounding and faithfulness gaps in long-form RAG, and it provides detailed quantitative improvements on a standard benchmark, making it the most informative read for the stated topic.

Themes

These papers converge on the problem of improving retrieval quality for RAG systems, especially when answering multi-hop or long-form queries. chATLAS shows that fine-tuning dense embeddings on domain-specific QA pairs and using a hybrid dense-sparse pipeline can more than double top-1 accuracy on a scientific internal corpus [P1]. On-policy state training tackles the same issue from a different angle: by exposing the retriever to its own mistaken states during training, it raises MuSiQue R@5 to 73.5 % and markedly improves chain coverage without changing the model architecture [P2]. Both approaches highlight that training on realistic retrieval trajectories--whether via domain adaptation or on-policy exposure--yields substantial gains over static, gold-only training.

A second shared theme is the need for coverage-aware selection to ensure that all sub-questions are grounded in evidence. Decompose-Estimate-Select builds a coverage-trained bi-encoder and a cross-encoder matrix to greedily pick documents that maximise sub-question coverage, boosting ?-nDCG@10 from 0.625 to 0.702 on TREC RAG 2024 and further improving vital-nugget coverage with a self-feedback loop [P3]. DeFT-RAG tackles multi-hop grounding by incrementally decomposing questions into self-contained sub-questions and using a lightweight decision model plus a QA cache to prune irrelevant passages, achieving state-of-the-art accuracy on MuSiQue, 2WikiMultiHopQA and HotpotQA while reducing LLM calls [P4]. Both systems demonstrate that explicit decomposition and coverage estimation outperform naïve relevance-based retrieval.

Evaluation methodology also emerges as a focal point. P3 and P4 rely on benchmark suites such as MuSiQue, TREC RAG 2024, 2WikiMultiHopQA and HotpotQA, but they expose different gaps: P3 notes that more retrieved evidence does not automatically improve answers, while P4 reports limited cache effectiveness on nested sub-questions. Complementing these, P5 proposes a sequential-decision framework that treats LLM-guided search as alternating "select" and "spawn" actions, showing that both the underlying retriever (e.g., SPLADE + Query Reformulation) and the policy materially affect recall on TREC Deep Learning 2019 [P5]. This work underscores the importance of jointly modelling retrieval and LLM-driven exploration rather than evaluating them in isolation.

Overall, the collection illustrates a spectrum of RAG system design choices--from embedding fine-tuning and on-policy training to coverage-aware document selection and interactive search policies--each validated on distinct benchmarks, and together they map current trade-offs between retrieval robustness, grounding fidelity, and evaluation rigor.

5 theme sentence(s) checked against the paper summaries.

At a glance

[P1] chATLAS: A Retrieval-Augmented Conversational Assistant for the ATLAS Experiment · relevance 0.52
chATLAS is a Retrieval-Augmented Generation assistant that gives ATLAS collaborators conversational access to >30,000 internal documents, achieving over a 140 % boost in top-1 retrieval accuracy when

using fine-tuned embeddings with contextual retrieval.

[P2] Retrieving from Your Own Mistakes: On-Policy Training of Embedding Models for Multi-Hop Retrieval-Augmented Generation · relevance 0.46
On-policy state training lets a chain-embedding retriever learn from its own mistaken retrieval states, markedly improving multi-hop evidence coverage and transferring to longer chains without extra training.

[P3] Decompose, Estimate, Select: Coverage-Aware Context Construction for Long-Form RAG · relevance 0.45
A coverage-aware retrieval pipeline that decomposes queries, predicts document-subquestion coverage, and selects documents greedily raises ?-nDCG@10 from 0.625 to 0.702 on TREC RAG 2024, and a grounded self-feedback loop further improves vital-nugget coverage by 0.086 without extra retrieval.

[P4] DeFT-RAG: Incrementally Decomposing Multi-Hop Questions into Self-Contained Sub-Questions with Reusable Answers · relevance 0.44
DeFT-RAG is a retrieval-augmented generation system that incrementally breaks multi-hop questions into self-contained sub-questions, uses rule-based QA tables and a lightweight decision model, and caches answers, achieving state-of-the-art accuracy with fewer LLM calls.

[P5] A Sequential Decision Framework for Evaluating LLM-Guided Search · relevance 0.42
The paper proposes a simple sequential-decision framework that models LLM-guided high-recall search as alternating "select" (exploiting a ranking) and "spawn" (creating a new ranking), and shows that both the underlying retriever and the policy materially affect recall performance.

[P1] chATLAS: A Retrieval-Augmented Conversational Assistant for the ATLAS Experiment

Joe Egan, Ben Elliot, Samuel Van Stroud, Alex Sopio et al. · arXiv 2610.37697v1 · https://arxiv.org/abs/2610.37697v1 · relevance 0.52

TL;DR: chATLAS is a Retrieval-Augmented Generation assistant that gives ATLAS collaborators conversational access to >30,000 internal documents, achieving over a 140 % boost in top-1 retrieval accuracy when using fine-tuned embeddings with contextual retrieval.

Problem
Large scientific collaborations such as ATLAS maintain heterogeneous, poorly searchable internal documentation, making it hard for members to locate and reuse knowledge [1][3].

Method
chATLAS combines a hybrid dense-sparse retrieval pipeline with a parent-child chunking scheme (1024-token parents split into four 256-token children) and a domain-adapted embedding model fine-tuned on ATLAS question-answer pairs; queries are routed through intent-screening, safety checks, and lightweight LLM normalization before vector search in a PostgreSQL + pgvector store, with results merged by Reciprocal Rank Fusion [5][6].

Key results
- The corpus comprises 4,450 recent TWiki pages, 9,982 MkDocs pages, 1,914 ATLAS papers, 1,250 CONF notes, 39 PUB notes, 217 lecture entries, and 13,824 Q&A threads [5].
- Fine-tuning the embedding model together with contextual retrieval more than doubles the frequency that the correct document appears as the top result on the most technically challenging source [1].
- On the TWiki benchmark, this combination improves top-1 retrieval accuracy by >140 % relative to the baseline model and yields the highest top-k accuracy across k = 1-10 [6].
- Retrieval uses approximately 20 dense candidates and 10 sparse candidates per query, with IVFFlat selected as the default ANN index for efficiency [6].

Limitations
The excerpts do not provide explicit statements about the system's limitations or failure cases.

Sources

[1] arXiv 2610.37697v1.pdf, arXiv abstract

[3] arXiv 2610.37697v1.pdf, page 6

[5] arXiv 2610.37697v1.pdf, page 10

[6] arXiv 2610.37697v1.pdf, page 15

[P2] Retrieving from Your Own Mistakes: On-Policy Training of Embedding Models for Multi-Hop Retrieval-Augmented Generation

Chenjie Qiu, Zixuan Zeng, Shaobin Shi, Bingfeng Chen et al. · arXiv 2610.40409v1 · https://arxiv.org/abs/2610.40409v1 · relevance 0.46

TL;DR: On-policy state training lets a chain-embedding retriever learn from its own mistaken retrieval states, markedly improving multi-hop evidence coverage and transferring to longer chains without extra training.

Problem
Complex questions often need evidence from several documents, and a retriever must collect all relevant passages to answer reliably [1]. Existing chain embedding retrievers are trained only on gold (perfect) histories, but at test time they condition on their own retrievals, leading to many mistakes: by the third step, about half or more of visited states contain a wrong passage, which dramatically reduces the chance of later retrieving missing gold passages [1][7].

Method
The authors introduce on-policy state training, which augments teacher-forced training with states actually visited by the current encoder [3]. At each epoch the encoder rolls out its greedy policy on all training questions, records each visited state (excluding duplicates), and labels it with the earliest missing gold passage while masking other missing gold passages [3]. The training objective mixes half teacher-forced gold states and half visited states (? = ½) and updates the encoder contrastively [3]. This procedure follows a DAgger-style dynamic oracle without changing the model architecture, loss, negatives, or index [3][4].

Key results
- On the MuSiQue benchmark, on-policy state training achieves R@5 = 73.5 % and R@10 = 80.5 %, with complete-chain coverage C@5 = 49.3 % and C@10 = 59.8 %, outperforming all listed baselines (e.g., Search-R1: R@5 = 66.7 %, C@10 = 49.5 %) [5].

- The retriever finds a missing gold passage in 67 % of clean states but only 28 % of contaminated ones, illustrating the impact of mistakes [3].
- After a miss, the next-step hit rate rises from 23 % (teacher forcing) to 34 % with on-policy training, showing better recovery from errors [8].
- Synthetic contamination of gold histories closes about half the performance gap to full on-policy training, indicating that part of the gain comes from exposure to mistaken histories [1].

Limitations
The excerpts do not detail explicit limitations of the approach; the paper does not report any major drawbacks beyond noting that some baseline comparisons (HippoRAG 2 and G-Retriever) were run on reduced corpora, which may affect the fairness of those comparisons [5].

Sources

[1] arXiv 2610.40409v1.pdf, arXiv abstract

[3] arXiv 2610.40409v1.pdf, page 4

[4] arXiv 2610.40409v1.pdf, page 4

[5] arXiv 2610.40409v1.pdf, page 6

[7] arXiv 2610.40409v1.pdf, page 9

[8] arXiv 2610.40409v1.pdf, page 13

[P3] Decompose, Estimate, Select: Coverage-Aware Context Construction for Long-Form RAG

He Tu, Aleena Bosco Kurumthottam, Torsten Suel · arXiv 2610.40314v1 · https://arxiv.org/abs/2610.40314v1 · relevance 0.45

TL;DR: A coverage-aware retrieval pipeline that decomposes queries, predicts document-subquestion coverage, and selects documents greedily raises ?-nDCG@10 from 0.625 to 0.702 on TREC RAG 2024, and a grounded self-feedback loop further improves vital-nugget coverage by 0.086 without extra retrieval.

Problem
Long-form retrieval-augmented generation (RAG) must answer multi-faceted queries that require synthesizing many distinct facts, yet relevance-based retrieval often returns redundant evidence and leaves some aspects unsupported [1][3]. Existing diversification methods do not capture nugget-level redundancy, and bi-encoders score documents independently, missing set-level objectives [2].

Method
The authors build a pipeline that (1) decomposes each query into subquestions using an LLM, (2) ranks candidate documents with a coverage-trained bi-encoder, (3) predicts a document × subquestion coverage matrix with a cross-encoder, and (4) selects documents via greedy maximum-marginal coverage [2][6]. The pipeline is trained on document-subquestion pairs from the Researchy Questions dataset using LLM answerability judgments and the CoveR contrastive coverage objective [2]. To address under-used evidence, a grounded self-feedback loop lets the generator identify unused evidence from its first answer and regenerate without gold supervision [1][2][7].

Key results
- On TREC RAG 2024, ?-nDCG@10 improves from 0.625 to 0.702, approaching the 0.718 upper bound [1][2].
- A selector with gold document-nugget labels attains 0.858 subtopic recall (StRecall) with five documents, the same as with ten, while a strong coverage-trained retriever reaches 0.762 with ten [2].
- The grounded self-feedback loop raises strict vital-nugget coverage (Vstrict) by 0.086 over two passes [1][2][7].
- On the CRUX-DUC04 collection, one pass of the loop yields a transfer-rate gain of +0.062 (p = .006) and an answer StRecall gain of +0.039 (p = .005); pooled across both collections (n = 104) the gains survive Holm correction (+0.056 and +0.030, both p = .007) [8].
- An oracle selector with substantially higher retrieved coverage does not improve any downstream answer metrics, indicating that more retrieved evidence does not automatically translate to better answers [7].

Limitations
The evaluation uses only two small collections (56 and 50 queries) [4][8]; ? = 0.3 was chosen on the evaluation set rather than a held-out set, making it optimistic [8]. No significance tests were run for Table 1, so its ordering is indicative only [8]. The best configuration relies on a proprietary decomposer, though an open Llama-3.3-70B-Instruct variant incurs only a modest ?-nDCG@10 drop (0.683 vs 0.702) [8]. Nugget links are LLM-judged, which may bias absolute scores, and the bi-encoder was trained on only 1,929 examples [8]. Multi-pass feedback was evaluated only on TREC RAG 2024, and Vstrict cannot be pooled for CRUX-DUC04 because it lacks importance labels [8].

Sources

[1] arXiv 2610.40314v1.pdf, arXiv abstract

[2] arXiv 2610.40314v1.pdf, page 2

[3] arXiv 2610.40314v1.pdf, page 3

[4] arXiv 2610.40314v1.pdf, page 4

[6] arXiv 2610.40314v1.pdf, page 7

[7] arXiv 2610.40314v1.pdf, page 11

[8] arXiv 2610.40314v1.pdf, page 12

[P4] DeFT-RAG: Incrementally Decomposing Multi-Hop Questions into Self-Contained Sub-Questions with Reusable Answers

Dingyi Kang, Bingzhe Li · arXiv 2610.37826v1 · https://arxiv.org/abs/2610.37826v1 · relevance 0.44

TL;DR: DeFT-RAG is a retrieval-augmented generation system that incrementally breaks multi-hop questions into self-contained sub-questions, uses rule-based QA tables and a lightweight decision model, and caches answers, achieving state-of-the-art accuracy with fewer LLM calls.

Problem
Retrieval-augmented generation (RAG) struggles with multi-hop questions because the required evidence chain often involves bridge entities that are not mentioned in the question, leading graph-based methods to miss parts of the chain and iterative methods to suffer from planning, dependency, and tracking errors[1][2].

Method
DeFT-RAG addresses these issues with three components: (1) an incremental decomposition that generates one self-contained sub-question at a time, each naming all its entities, eliminating the need for planning or rewriting; (2) rule-based updates to a QA table and yes/no decisions made by a fine-tuned lightweight decision model (DM) that handle tracking and relevance outside the LLM; and (3) a QA cache that reuses answers for identical sub-questions across questions, reducing retrieval and LLM calls[1][2][3][4].

Key results
- On MuSiQue, 2WikiMultiHopQA and HotpotQA, DeFT-RAG attains the highest accuracy among thirteen baselines with both gpt-4o-mini and Qwen3.6 backbones, while using fewer input tokens than any other iterative method[1].
- Gains on MuSiQue are +5.2 points over the second-best Hi-Q with gpt-4o-mini (0.629 vs. 0.577) and +5.3 points with Qwen3.6 (0.636 vs. 0.583)[5].
- On 2Wiki, DeFT-RAG is 2.9 points ahead of LogicRAG with Qwen3.6 and matches Self-Ask with gpt-4o-mini (0.852 vs. 0.849)[5].
- On HotpotQA, it is 3.4 points ahead of Hi-Q with gpt-4o-mini and comparable to Hi-Q and LinearRAG with Qwen3.6 (0.871 vs. 0.864 and 0.857)[5].
- With a QA cache shared across 1,000 MuSiQue questions, 23.1 % of sub-questions are answered from the cache, cutting LLM calls by 12.9 % without lowering accuracy[1].
- Ablating passage selection reduces accuracy by 2.3, 3.7 and 5.3 points on MuSiQue, 2Wiki and HotpotQA respectively, confirming the importance of the DM's relevance filtering[6].
- Fine-tuning the DM raises end-to-end accuracy on the first 300 MuSiQue questions from 0.587 to 0.623, mainly by improving the direct-answer decision[6].

Limitations
DeFT-RAG can fail on nested sub-questions: on 2Wiki bridge-comparison questions, 38 % of such sub-questions receive no answer, and many reach the step limit, which explains why DeFT-RAG only matches Self-Ask on 2Wiki with gpt-4o-mini[8]. Errors also arise from the backbone LLM, such as asking for the wrong fact (e.g., retrieving a birth date instead of a signing date) and from decision-model mistakes, though the latter account for only about 19 of 148 wrong answers on 2Wiki with gpt-4o-mini[8].

Sources

[1] arXiv 2610.37826v1.pdf, arXiv abstract

[2] arXiv 2610.37826v1.pdf, page 2

[3] arXiv 2610.37826v1.pdf, page 4

[4] arXiv 2610.37826v1.pdf, page 5

[5] arXiv 2610.37826v1.pdf, page 6

[6] arXiv 2610.37826v1.pdf, page 7

[8] arXiv 2610.37826v1.pdf, page 12

[P5] A Sequential Decision Framework for Evaluating LLM-Guided Search

Siqing Huo, Andrew Parry, Debasis Ganguly, Mark Smucker et al. · arXiv 2610.39231v1 · https://arxiv.org/abs/2610.39231v1 · relevance 0.42

TL;DR: The paper proposes a simple sequential-decision framework that models LLM-guided high-recall search as alternating "select" (exploiting a ranking) and "spawn" (creating a new ranking), and shows that both the underlying retriever and the policy materially affect recall performance.

Problem
The authors target high-recall information retrieval (HRIR), where the goal is to find as many relevant documents as possible within a limited judging budget [J] [2]. Existing retrieval strategies can be expressed within a unified framework that compares policies that iteratively generate queries, retrieve documents, and use LLM judgments to guide further search [1].

Method
They formalize the search environment as a set of ranked lists and define two primitive actions: **select**, which advances judgment through an existing list, and **spawn**, which creates a new list via query generation or reformulation [3]. A policy ? maps the current state to either action, aiming to maximize a cumulative reward (e.g., relevance-based reward) over the search trajectory [3]. The framework can express classic interactive search, continuous active learning, and LLM-generated query reformulation as special cases [4]. Experiments instantiate several policies (Adhoc, Query-Reformulation + MTF, Forward-Only Spawning, Bounded Spawning) across four retrieval pipelines (BM25, SPLADE-v3, TCT-ColBERT-v2, a hybrid BM25 + SPLADE + TCT-ColBERT reranked by MonoT5/DuoT5) on the TREC Deep Learning 2019 collection, using GPT-4o-mini for relevance judgments and GPT-5 for spawning [5][6].

Key results
- Recall@R is reported for all 16 runs (4 pipelines × 4 policies); the best overall policy is **SPLADE + Query Reformulation + MTF**, which "substantially outperforms every policy using BM25 as its sole ranking method, with no overlap in the reported 95 % bootstrap confidence intervals" [6].
- The experiments demonstrate that both the choice of underlying retriever and the exploration policy "materially affect performance" [6].
- No explicit numeric recall scores or percentage gains are provided in the excerpts; results are presented only as comparative statements and confidence-interval observations [6].

Limitations
- The paper evaluates using a normalized judging budget J = R (recall@R), which "does not represent viable stopping rules in practice" because it assumes knowledge of the total number of relevant documents [2].
- Cost modeling treats each LLM judgment as a unit cost and ignores variations in prompt length, retrieval/reranking costs, and other practical overheads [3].
- Concrete performance numbers (e.g., exact recall percentages) are not included in the provided excerpts, limiting quantitative assessment of the gains [6].

- The study is confined to the TREC DL 2019 collection and specific LLMs (GPT-4o-mini, GPT-5), so generalizability to other domains or models is not demonstrated in the excerpts [5][6].

Sources

[1] arXiv 2610.39231v1.pdf, arXiv abstract

[2] arXiv 2610.39231v1.pdf, page 2

[3] arXiv 2610.39231v1.pdf, page 3

[4] arXiv 2610.39231v1.pdf, page 5

[5] arXiv 2610.39231v1.pdf, page 6

[6] arXiv 2610.39231v1.pdf, page 6